L'Architecture Client-Serveur et le Protocole HTTP

Thème principal : Fonctionnement du Web, Architecture Client-Serveur et Protocole HTTP

Synthèse exécutive : Ce cours formalise le modèle d'interaction asymétrique Client-Serveur et analyse le protocole HTTP (HyperText Transfer Protocol), standard universel d'échange de ressources sur le Web. L'étudiant y découvre la nature 'stateless' (sans état) de HTTP et les mécanismes de contournement (cookies/sessions), le formatage textuel rigoureux des requêtes et des réponses (lignes de commande, en-têtes headers, corps body), la distinction fonctionnelle et sécuritaire entre les méthodes GET et POST, ainsi que la typologie des codes de statut HTTP (2xx, 3xx, 4xx, 5xx) indispensable au débogage et à l'architecture applicative backend/frontend.

L'Architecture Client-Serveur et la Nature 'Stateless' de HTTP

L'architecture client-serveur régit l'intégralité des échanges sur le réseau Web. Contrairement aux réseaux Pair-à-Pair (P2P) où chaque machine est à la fois cliente et serveuse, le Web repose sur un modèle strictement asymétrique. Le client (un navigateur web tel que Chrome ou Firefox) émet une requête HTTP à destination d'un serveur web (processus applicatif distant comme Apache, Nginx ou IIS). Le serveur écoute en permanence sur un port réseau standardisé (port 80 pour HTTP en clair, port 443 pour HTTPS chiffré), traite la demande et retourne un message de réponse HTTP. Le protocole HTTP est dit 'stateless' (sans état) : une fois la réponse transmise, la connexion est fermée ou réinitialisée, et le serveur oublie instantanément le client. Cette absence de mémoire garantit une excellente scalabilité (les serveurs n'ont pas à maintenir en mémoire vive des millions d'états de connexion active), mais interdit le fonctionnement natif d'applications complexes comme des boutiques e-commerce. Pour pallier cette contrainte, on utilise des cookies : le serveur transmet un identifiant de session dans un en-tête 'Set-Cookie', que le navigateur enregistre et renvoie automatiquement dans les en-têtes de toutes les requêtes ultérieures.

- Asymétrie Client-Serveur : Rôles stricts entre le client actif qui demande et le serveur passif qui écoute.
- Protocole Stateless (Sans état) : Absence de mémoire des requêtes précédentes par le protocole HTTP.
- Mécanismes de Session : Utilisation de cookies et de sessions côté serveur pour simuler un état persistant.

Cas concret enseignant : *Le professeur compare HTTP à un guichetier amnésique : chaque fois que vous vous présentez devant lui, même si c'était il y a deux secondes, il vous traite comme un inconnu total. Pour qu'il se souvienne de vous, vous devez lui montrer votre badge (le cookie de session) à chaque phrase. Il ajoute qu'en chargeant la page d'accueil de l'IUT de Bayonne, le navigateur n'envoie pas une seule requête unique, mais des dizaines de requêtes HTTP simultanées pour récupérer séparément le HTML, les feuilles de style CSS, les scripts JS et les images.*

Structure des Messages HTTP et Comparatif GET vs POST

Un message HTTP transite sous forme textuelle brute. Une requête comporte : 1. La ligne de requête (verbe HTTP, URI de la ressource, version du protocole) ; 2. Les en-têtes (clé-valeur fournissant le contexte) ; 3. Une ligne vide servant de séparateur strict ; 4. Le corps (Body) contenant les données. La réponse du serveur adopte la même structure mais commence par une ligne de statut (ex: 'HTTP/1.1 200 OK'). Le choix entre les verbes GET et POST est crucial : GET sert à lire des données sans modifier le serveur (idempotence). Ses paramètres sont accolés à l'URL. En raison de cette visibilité, GET ne doit JAMAIS transporter de données sensibles. POST est utilisé pour soumettre des données altérant le système. Les paramètres transitent dans le corps (Body) de la requête, permettant de transférer de grands volumes d'informations et évitant l'affichage des variables dans la barre d'adresse.

- Message Textuel Normalisé : Requête et réponse structurées en texte clair avec une première ligne, des en-têtes (headers), une ligne vide et un corps (body).
- Méthode GET : Lecture et consultation de ressources, paramètres visibles dans l'URL, méthode sûre et idempotente.
- Méthode POST : Modification de l'état du serveur, paramètres encapsulés dans le corps de la requête.

Cas concret enseignant : *Le professeur met en garde contre l'utilisation absurde de GET pour transmettre un mot de passe : soumettre un mot de passe en GET l'affiche en clair dans l'URL de l'écran, le sauvegarde dans l'historique du navigateur et l'enregistre en clair dans les fichiers de logs du serveur web. C'est la vulnérabilité parfaite au 'shoulder surfing' (regard par-dessus l'épaule).*

Décodage et Classification des Codes de Statut HTTP

Chaque réponse émise par le serveur inclut un code numérique à trois chiffres permettant au navigateur ou à l'application cliente d'interpréter instantanément l'issue de la demande : - 2xx (Succès) : La requête a été reçue, comprise et acceptée (ex: 200 OK, 201 Created). - 3xx (Redirection) : Indique que le client doit émettre une nouvelle requête vers un autre emplacement (ex: 301 Moved Permanently, 304 Not Modified). - 4xx (Erreur Client) : La requête contient une anomalie. Exemples : 401 (Authentification requise), 403 (Forbidden - accès refusé), 404 (Not Found - ressource absente). - 5xx (Erreur Serveur) : La demande du client est valide mais le serveur a échoué lors de son exécution interne. Exemples : 500 (Internal Server Error - plantage de script), 503 (Service Unavailable - serveur saturé).

- Ligne de Statut de Réponse : Première ligne d'une réponse HTTP indiquant le résultat du traitement à l'aide d'un code à 3 chiffres.
- Familles de Codes : 2xx (Succès), 3xx (Redirection), 4xx (Erreur Client), 5xx (Erreur Serveur).
- Frontière de Responsabilité : Distinguer l'erreur imputable au client (4xx) du dysfonctionnement interne du serveur (5xx).

Cas concret enseignant : *Le professeur raconte une anecdote personnelle : à ses débuts, une erreur de syntaxe dans son script PHP (un simple point-virgule manquant) faisait planter la connexion à la base de données lors de la validation d'un formulaire. Le serveur renvoyait systématiquement une erreur 500 Internal Server Error. Les utilisateurs paniquaient en pensant que le site était piraté alors qu'il s'agissait simplement d'une faute de frappe dans son code backend.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Monsieur, quelle est la différence concrète entre GET et POST pour envoyer un formulaire de connexion ?
Réponse du professeur : GET transmet les paramètres directement dans l'URL, ce qui les rend visibles dans la barre d'adresse et stockés dans l'historique (à proscrire absolument pour les mots de passe). POST encapsule les données dans le corps de la requête, les masquant de la barre d'adresse, tout en nécessitant impérativement HTTPS pour chiffrer le contenu sur le réseau.

■ Exercice & Démonstration d'Application

Énoncé : Analyser la requête HTTP brute suivante. Identifier la méthode utilisée, la ressource demandée, l'hôte cible, le rôle de l'agent utilisateur, et expliquer le comportement exact du serveur avec son code de statut si la ressource est inexistante : GET /images/logo.png HTTP/1.1 Host: iutbayonne.univ-pau.fr User-Agent: Mozilla/5.0

Résolution pas à pas :
1. Analyse de la ligne de requête ('GET /images/logo.png HTTP/1.1') : - Méthode HTTP : GET (extraction/récupération de ressource sans modification de l'état du serveur). - Ressource sollicitée (URI) : '/images/logo.png' (chemin absolu vers le fichier image). - Version du protocole : HTTP/1.1. 2. Analyse de l'en-tête 'Host' ('Host: iutbayonne.univ-pau.fr') : - Spécifie le nom de domaine du serveur destinataire, obligatoire en HTTP/1.1 pour le 'Virtual Hosting' (hébergement de plusieurs sites sur une unique IP). 3. Analyse de l'en-tête 'User-Agent' ('User-Agent: Mozilla/5.0') : - Transmet la signature du logiciel client (navigateur) émettant la requête. 4. Scénario d'absence de la ressource : - Si le fichier 'logo.png' n'existe pas dans le répertoire '/images/' du serveur web, la demande pointe vers un emplacement invalide. - L'erreur provenant d'une URL erronée formulée par le client, la responsabilité lui incombe. - Le serveur retourne un code de statut de la classe 4xx, précisément le code '404 Not Found'.

Résultat : Méthode : GET | Ressource : /images/logo.png | Hôte cible : iutbayonne.univ-pau.fr | Rôle User-Agent : Identification du client | Code en cas d'absence : 404 Not Found (Erreur sous la responsabilité

du client).

■ Auto-Évaluation Flash d'Examen

Q : Pourquoi qualifie-t-on le protocole HTTP de 'stateless' (sans état) et quelle est sa conséquence directe ?

R : HTTP est sans état car chaque transaction requête-réponse est totalement indépendante et amnésique. Le serveur ne conserve aucune mémoire des requêtes passées, ce qui nécessite d'utiliser des cookies et des sessions pour maintenir un suivi utilisateur comme un panier d'achat.

Q : Quelle est la différence fondamentale de responsabilité entre une erreur HTTP de la série 4xx et une erreur de la série 5xx ?

R : Une erreur 4xx indique un problème imputable au client (URL erronée, syntaxe incorrecte, absence d'autorisation). Une erreur 5xx indique que la requête était valide, mais que le serveur a rencontré une défaillance interne (crash de script, base de données inaccessible).

Q : Pourquoi la méthode HTTP GET ne doit-elle jamais être employée pour transmettre des données sensibles comme un mot de passe ?

R : Parce que GET transmet les paramètres directement dans l'URL en clair. Ces données sont ainsi affichées à l'écran, enregistrées dans l'historique du navigateur et stockées dans les fichiers de logs du serveur, créant une faille critique de sécurité.